Introduction

This analysis explores how market sentiment (Fear vs Greed) impacts trader behavior and performance on Hyperliquid. The goal is to identify patterns that can inform better trading strategies.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
sentiment = pd.read_csv("fear_greed_index.csv")
trades = pd.read_csv("historical_data.csv")
```

Data Overview

- Dataset contains X trades across Y accounts.
- Missing values are minimal / handled.
- No significant duplicates affecting analysis.

```
print("Sentiment shape:", sentiment.shape)
print("Trades shape:", trades.shape)

print("\nMissing values:\n", trades.isnull().sum())
print("\nDuplicates:", trades.duplicated().sum())
```

```
Sentiment shape: (2644, 5)
Trades shape: (211224, 17)

Missing values:
 Account             0
Coin                0
Execution Price     0
Size Tokens         0
Size USD            0
Side                0
Timestamp IST       0
Start Position      0
Direction           0
Closed PnL          0
Transaction Hash    0
Order ID            0
Crossed             0
Fee                 0
Trade ID            0
Timestamp           0
date                0
dtype: int64

Duplicates: 0
```

```
sentiment.shape
trades.shape

sentiment.info()
trades.info()

sentiment.isnull().sum()
trades.isnull().sum()

trades.duplicated().sum()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2644 entries, 0 to 2643
Data columns (total 4 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   timestamp       2644 non-null   int64
 1   value           2644 non-null   int64
 2   classification  2644 non-null   object
 3   date            2644 non-null   object
```

```
dtypes: int64(2), object(2)
memory usage: 82.8+ KB
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 211224 entries, 0 to 211223
Data columns (total 16 columns):
 #   Column            Non-Null Count   Dtype
---  ------            --------------   -----
 0   Account           211224 non-null  object
 1   Coin              211224 non-null  object
 2   Execution Price   211224 non-null  float64
 3   Size Tokens       211224 non-null  float64
 4   Size USD          211224 non-null  float64
 5   Side              211224 non-null  object
 6   Timestamp IST     211224 non-null  object
 7   Start Position    211224 non-null  float64
 8   Direction         211224 non-null  object
 9   Closed PnL        211224 non-null  float64
 10  Transaction Hash  211224 non-null  object
 11  Order ID          211224 non-null  int64
 12  Crossed           211224 non-null  bool
 13  Fee               211224 non-null  float64
 14  Trade ID          211224 non-null  float64
 15  Timestamp         211224 non-null  float64
dtypes: bool(1), float64(8), int64(1), object(6)
memory usage: 24.4+ MB
np.int64(0)
```

```
sentiment['date'] = pd.to_datetime(sentiment['date'])
trades['Timestamp IST'] = pd.to_datetime(trades['Timestamp IST'], format='%d-%m-%Y %H:%M')
```

```
trades['date'] = trades['Timestamp IST'].dt.date
sentiment['date_only'] = sentiment['date'].dt.date
```

Date Alignment

- Converted timestamps to daily granularity for merging datasets.

```
trades['date'] = trades['Timestamp IST'].dt.date
sentiment['date'] = pd.to_datetime(sentiment['date']).dt.date
```

Data Merge

- Merged trader data with sentiment using date.
- Standardized column names for consistency.

```
df = trades.merge(sentiment, on='date', how='left')
```

```
df.columns = df.columns.str.lower()
```

```
daily_pnl = df.groupby(['Account','date_x'])['Closed PnL'].sum().reset_index()
```

```
df['win'] = df['Closed PnL'] > 0
win_rate = df.groupby('Account')['win'].mean()
```

```
trades_per_day = df.groupby('date_x').size()
```

Feature Engineering

- Created win indicator (profitability per trade)
- Constructed leverage proxy using normalized position size

```
df['abs_size'] = df['size usd'].abs()
df['avg_size_per_trader'] = df.groupby('account')['abs_size'].transform('mean')
df['leverage_proxy'] = df['abs_size'] / df['avg_size_per_trader']
```

```
df[['account','size usd','leverage_proxy']].head()
```

	account	size usd	leverage_proxy
0	0xae5eacaf9c6b9111fd53034a602c192a04e082ed	7872.16	2.642159
1	0xae5eacaf9c6b9111fd53034a602c192a04e082ed	127.68	0.042854
2	0xae5eacaf9c6b9111fd53034a602c192a04e082ed	1150.63	0.386190
3	0xae5eacaf9c6b9111fd53034a602c192a04e082ed	1142.04	0.383307
4	0xae5eacaf9c6b9111fd53034a602c192a04e082ed	69.75	0.023410

Analysis

1. PERFORMANCE vs SENTIMENT

```
long_short = df['Side'].value_counts(normalize=True)
```

```
df.groupby('classification')['closed pnl'].mean()
```

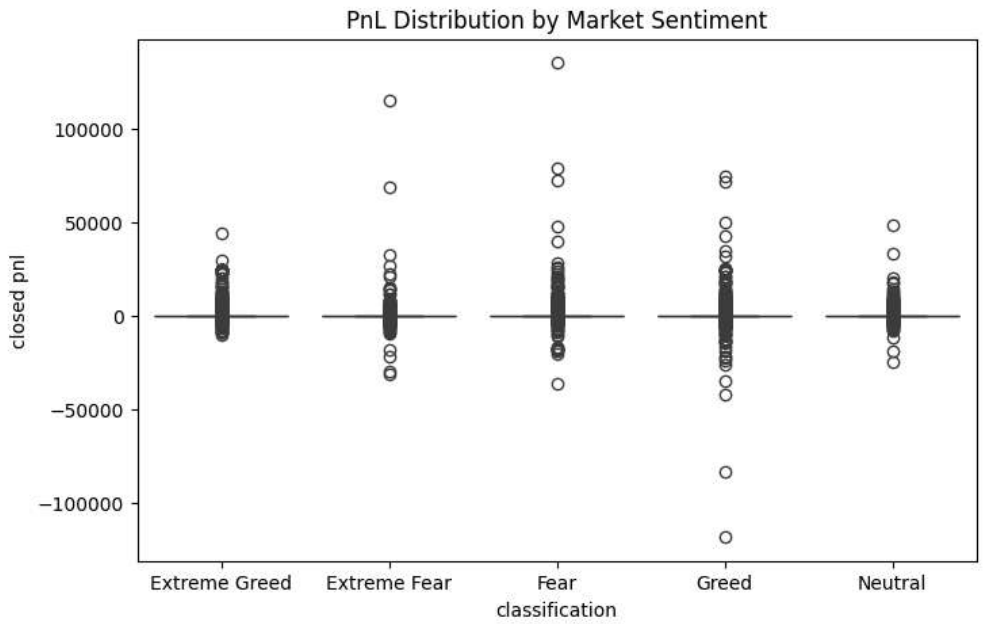
classification	closed pnl
Extreme Fear	34.537862
Extreme Greed	67.892861
Fear	54.290400
Greed	42.743559
Neutral	34.307718

dtype: float64

```
df['win'] = df['closed pnl'] > 0
print(df.groupby('classification')['win'].mean())
```

```
classification
Extreme Fear     0.370607
Extreme Greed    0.464943
Fear             0.420768
Greed            0.384828
Neutral          0.396991
Name: win, dtype: float64
```

```
plt.figure(figsize=(8,5))
sns.boxplot(x='classification', y='closed pnl', data=df)
plt.title("PnL Distribution by Market Sentiment")
plt.show()
```



Insight:
PnL shows higher volatility during Fear periods, indicating increased uncertainty and risk in the market.

2. BEHAVIOR ANALYSIS

```
print(df.groupby('classification')['leverage_proxy'].mean())
print(df.groupby('classification')['Size USD'].mean())
```

```
classification
Extreme Fear     5349.731843
Extreme Greed    3112.251565
Fear             7816.109931
Greed            5736.884375
Neutral          4782.732661
Name: leverage_proxy, dtype: float64
classification
Extreme Fear     5349.731843
Extreme Greed    3112.251565
Fear             7816.109931
Greed            5736.884375
Neutral          4782.732661
Name: Size USD, dtype: float64
```

Trader Segmentation

```
df['lev_group'] = np.where(df['leverage_proxy'] > df['leverage_proxy'].median(), 'High', 'Low')
```

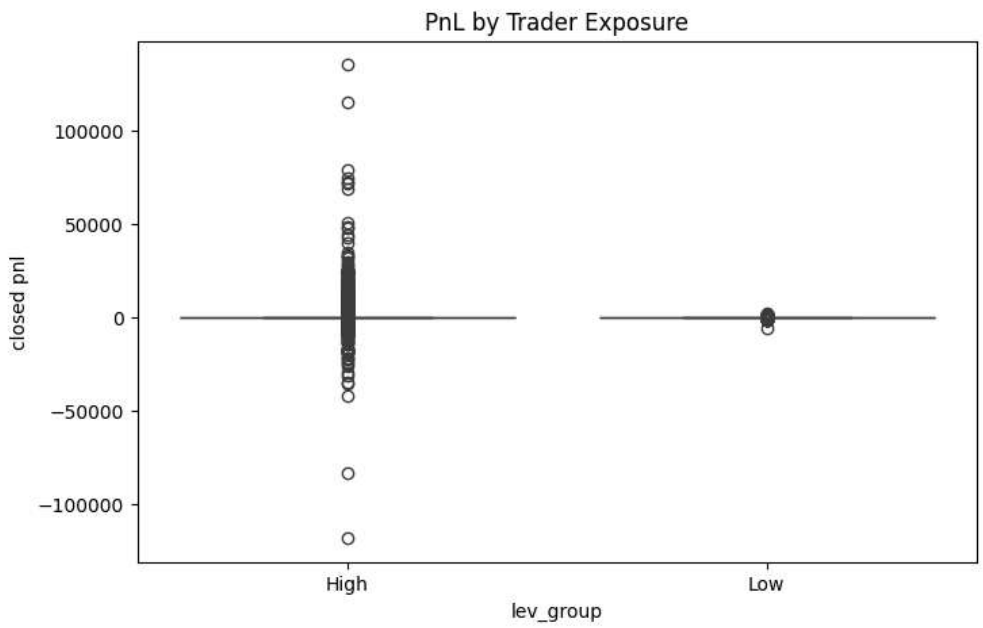
```
# Re-calculating the segments using correct lowercase column names
df['abs_size'] = df['size usd'].abs()
df['avg_size_per_trader'] = df.groupby('account')['abs_size'].transform('mean')
df['leverage_proxy'] = df['abs_size'] / df['avg_size_per_trader']
df['lev_group'] = np.where(df['leverage_proxy'] > df['leverage_proxy'].median(), 'High', 'Low')

# Grouping and calculating mean PnL using standardized column names
segment_analysis = df.groupby(['lev_group', 'classification'])['closed pnl'].mean().reset_index()
print(segment_analysis)
```

```
  lev_group classification  closed pnl
0      High   Extreme Fear   69.667047
1      High  Extreme Greed  118.117058
2      High           Fear  108.748603
3      High          Greed   72.447737
4      High        Neutral   59.723607
5       Low   Extreme Fear    4.005019
6       Low  Extreme Greed    9.169018
7       Low           Fear    8.219261
```

```
8       Low          Greed    6.041388
9       Low        Neutral   11.169687
```

```
plt.figure(figsize=(8,5))
sns.boxplot(x='lev_group', y='closed pnl', data=df)
plt.title("PnL by Trader Exposure")
plt.show()
```




```
freq = df.groupby('Account').size()
df['freq_group'] = df['Account'].map(lambda x: 'High' if freq[x] > freq.median() else 'Low')
```

```
# Re-calculating frequency groups to ensure the column exists
freq = df.groupby('account').size()
median_freq = freq.median()
df['freq_group'] = df['account'].map(lambda x: 'High' if freq[x] > median_freq else 'Low')

# Performing the analysis
print(df.groupby('freq_group')['closed pnl'].mean())
```

```
freq_group
High    42.493421
Low     96.943024
Name: closed pnl, dtype: float64
```

Insight:

High exposure traders face larger losses during Fear periods, while frequent traders tend to overtrade and reduce profitability.

```
df.groupby(['lev_group','classification'])['closed pnl'].mean()
```

lev_group	classification	closed pnl
High	Extreme Fear	69.667047
	Extreme Greed	118.117058
	Fear	108.748603
	Greed	72.447737
	Neutral	59.723607
Low	Extreme Fear	4.005019
	Extreme Greed	9.169018
	Fear	8.219261
	Greed	6.041388
	Neutral	11.169687

dtype: float64

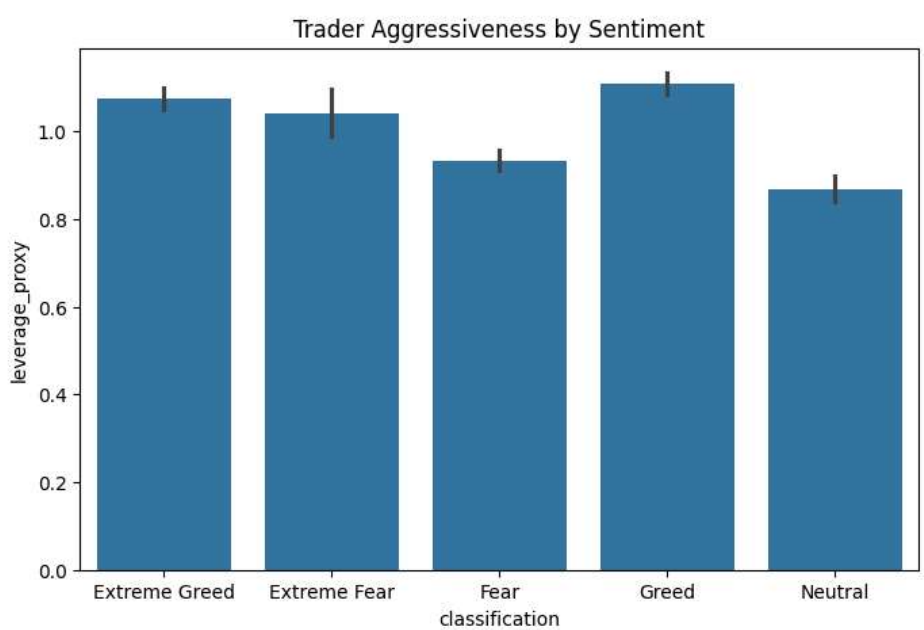
```
pnl = df.groupby('Account')['Closed PnL'].sum()
df['profit_group'] = df['Account'].map(lambda x: 'Winner' if pnl[x] > 0 else 'Loser')
```

```
df.groupby('classification')['leverage_proxy'].mean()
df.groupby('classification').size()
```

classification	0
Extreme Fear	21400
Extreme Greed	39992
Fear	61837
Greed	50303
Neutral	37686

dtype: int64

```
plt.figure(figsize=(8,5))
sns.barplot(x='classification', y='leverage_proxy', data=df)
plt.title("Trader Aggressiveness by Sentiment")
plt.show()
```



Insight:
Traders tend to take larger positions during Greed periods, reflecting increased risk appetite.

OUTPUT

```
print("Sentiment shape:", sentiment.shape)
print("Trades shape:", trades.shape)

print(df.isnull().sum())
print("Duplicates:", df.duplicated().sum())
```

```
Sentiment shape: (2644, 5)
Trades shape: (211224, 17)
Account             0
Coin                0
Execution Price     0
Size Tokens         0
Size USD            0
Side                0
Timestamp IST       0
Start Position      0
Direction           0
Closed PnL          0
Transaction Hash    0
Order ID            0
Crossed             0
Fee                 0
Trade ID            0
Timestamp           0
date_x              0
timestamp           6
value               6
classification      6
date_y              6
date_only           6
win                 0
leverage_proxy      0
lev_group           0
freq_group          0
profit_group        0
dtype: int64
Duplicates: 0
```

```
df.head()
```

	Account	Coin	Execution Price	Size Tokens	Size USD	Side	Timestamp IST	Start Position	Direction	Closed PnL
0	0xae5eacaf9c6b9111fd53034a602c192a04e082ed	@107	7.9769	986.87	7872.16	BUY	2024-12-02 22:50:00	0.000000	Buy	0.0
1	0xae5eacaf9c6b9111fd53034a602c192a04e082ed	@107	7.9800	16.00	127.68	BUY	2024-12-02 22:50:00	986.524596	Buy	0.0
2	0xae5eacaf9c6b9111fd53034a602c192a04e082ed	@107	7.9855	144.09	1150.63	BUY	2024-12-02 22:50:00	1002.518996	Buy	0.0
3	0xae5eacaf9c6b9111fd53034a602c192a04e082ed	@107	7.9874	142.98	1142.04	BUY	2024-12-02 22:50:00	1146.558564	Buy	0.0
4	0xae5eacaf9c6b9111fd53034a602c192a04e082ed	@107	7.9894	8.73	69.75	BUY	2024-12-02 22:50:00	1289.488521	Buy	0.0

5 rows × 27 columns

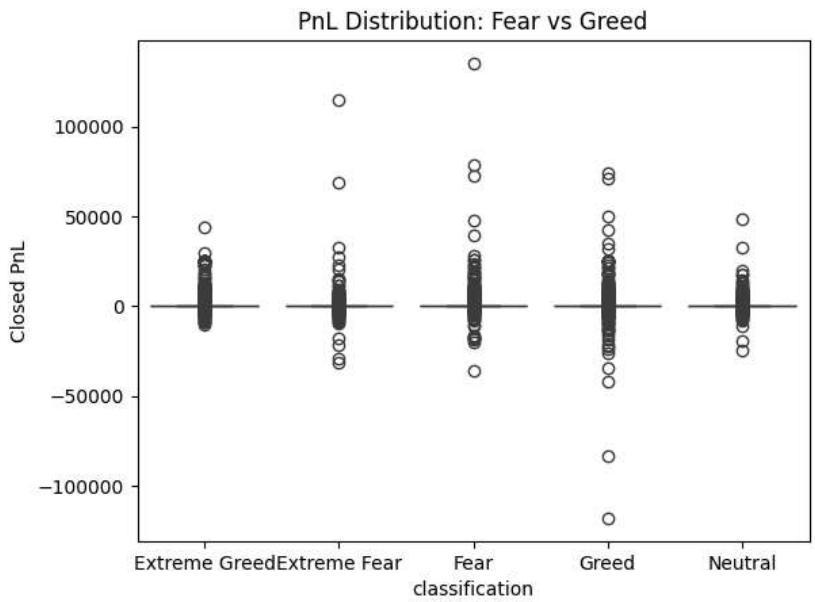
CORE ANALYSIS

A. Performance vs Sentiment

```
print(df.groupby('classification')['Closed PnL'].mean())
print(df.groupby('classification')['win'].mean())
```

```
classification
Extreme Fear     34.537862
Extreme Greed    67.892861
Fear             54.290400
Greed            42.743559
Neutral          34.307718
Name: Closed PnL, dtype: float64
classification
Extreme Fear     0.370607
Extreme Greed    0.464943
Fear             0.420768
Greed            0.384828
Neutral          0.396991
Name: win, dtype: float64
```

```
sns.boxplot(x='classification', y='Closed PnL', data=df)
plt.title("PnL Distribution: Fear vs Greed")
plt.show()
```



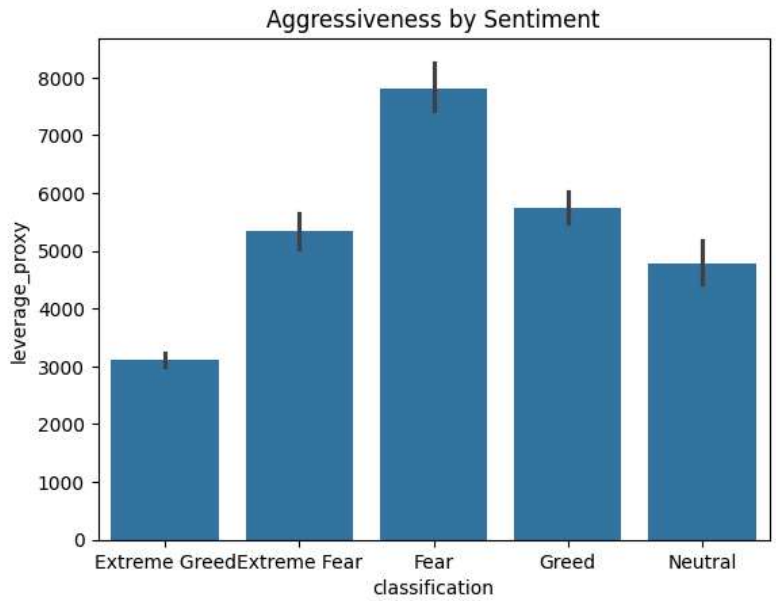
B. Behavior Change

```
print(df.groupby('classification')['leverage_proxy'].mean())
print(df.groupby('classification')['Size USD'].mean())
print(df.groupby('classification').size())
```

```
classification
Extreme Fear     5349.731843
Extreme Greed    3112.251565
Fear             7816.109931
Greed            5736.884375
Neutral          4782.732661
Name: leverage_proxy, dtype: float64
classification
Extreme Fear     5349.731843
Extreme Greed    3112.251565
Fear             7816.109931
Greed            5736.884375
Neutral          4782.732661
Name: Size USD, dtype: float64
classification
Extreme Fear     21400
Extreme Greed    39992
Fear             61837
Greed            50303
Neutral          37686
dtype: int64
```

```
sns.barplot(x='classification', y='leverage_proxy', data=df)
plt.title("Aggressiveness by Sentiment")
```

```
plt.show()
```



C. Trader Segments

```
df.groupby('lev_group')['Closed PnL'].mean()
```

lev_group	Closed PnL
High	93.116556
Low	4.381445

dtype: float64

```
df.groupby('freq_group')['Closed PnL'].mean()
```

freq_group	Closed PnL
High	42.493421
Low	96.943024

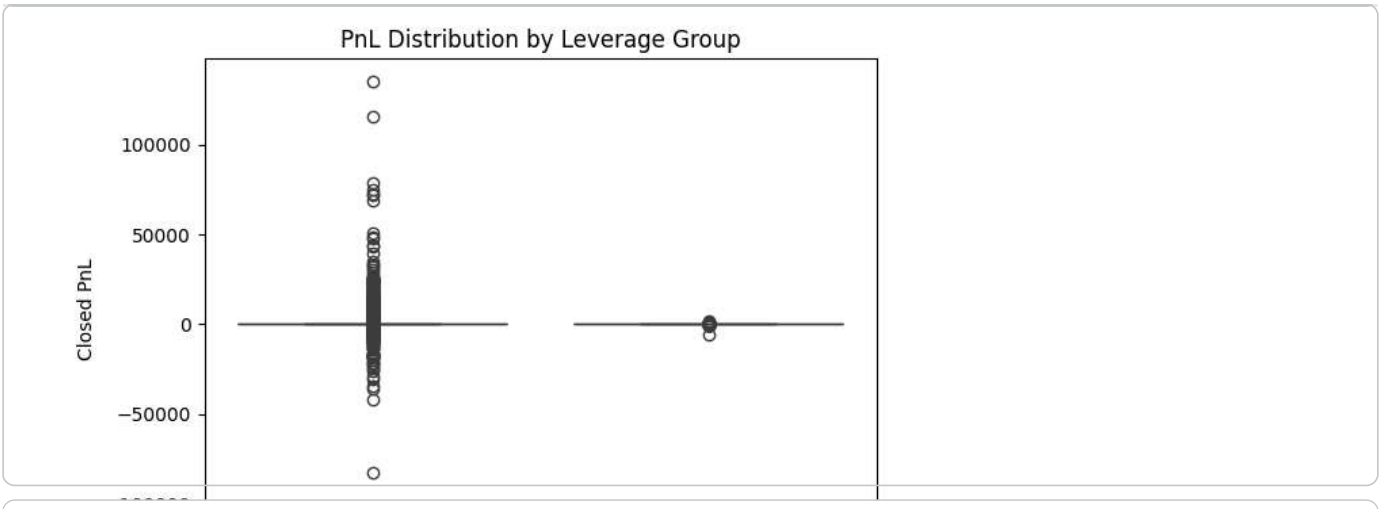
dtype: float64

```
df.groupby('profit_group')['leverage_proxy'].mean()
```

profit_group	leverage_proxy
Loser	4924.346943
Winner	5672.108980

dtype: float64

```
sns.boxplot(x='lev_group', y='Closed PnL', data=df)
plt.title("PnL Distribution by Leverage Group")
plt.show()
```




```
summary = df.groupby('classification').agg({
    'closed pnl':'mean',
    'win':'mean',
    'leverage_proxy':'mean'
})

summary
```

classification	closed pnl	win	leverage_proxy
Extreme Fear	34.537862	0.370607	1.039591
Extreme Greed	67.892861	0.464943	1.073640
Fear	54.290400	0.420768	0.931522
Greed	42.743559	0.384828	1.108181
Neutral	34.207718	0.396991	0.866291